From Prompting to Verification: How Experience Shapes Vibe Coding Practices

AHMED FAWZY, School of Mathematical and Computational Sciences, Massey University, New Zealand
AMJED TAHIR, School of Mathematical and Computational Sciences, Massey University, New Zealand
KELLY BLINCOE, Department of Electrical, Computer, and Software Engineering, University of Auckland, New Zealand

AI code generation tools have expanded software creation beyond professional developers, giving rise to *vibe coding*, a practice in which users generate software via natural-language prompts, evaluate outputs primarily by execution.Prior work has examined how AI code generation tools support programming tasks within specific user groups, typically professional developers, leaving open the question of how vibe coding practices differ across experience levels. We address this gap by surveying 162 vibe coders belonging to three user experience groups: non-coders, novices, and professional developers. Our results show that experience selectively shapes vibe coding. Reported experiences and perceptions of code quality are broadly similar across groups, with all three recognising both the strengths and limitations of vibe coding. In contrast, motivations, interaction styles, and quality assurance practices diverge with experience. Non-developers are most motivated by accessibility, novices emphasise learning and experimentation, and professionals use vibe coding more frequently in work-related contexts. We synthesise these findings as a *perception–action gap*: a general awareness of risks in AI-generated code is broadly distributed, but the capacity to evaluate, debug, and verify remains experience-dependent. We show that vibe coding is partially democratising as it broadens access to software creation without equally distributing the expertise to evaluate it.

CCS Concepts: • **Software and its engineering → Automatic programming**; • **Human-centered computing → Interactive systems and tools**.

1 Introduction

AI code generation tools, powered by large language models (LLMs), have rapidly become embedded in software development. Since the release of GitHub Copilot in 2021 and the public availability of conversational assistants such as ChatGPT in late 2022, these tools have evolved from inline autocomplete-style suggestions into interactive, multi-turn workflows in which coders describe functionality in natural language and iteratively refine AI-generated code through prompt–response cycles [5, 37]. A further shift occurred in early 2025 with the emergence of CLI-based agentic coding tools such as Cursor, Claude Code, and OpenAI Codex, which can autonomously plan, execute, and iterate on multi-step development tasks with limited manual intervention [28, 35]. Adoption has scaled accordingly. For example, the 2025 Stack Overflow Developer Survey reports that 84% of respondents use or plan to use AI code generation tools in their development process, up from 76% in 2024, with 51% of professional developers use them daily [66]. Empirical studies report measurable productivity gains for routine programming tasks [57, 82], although recent randomised evidence with experienced open-source developers finds that perceived productivity gains do not always match objective task completion times [6]. Persistent concerns also remain regarding code quality [79], security [27, 63], overconfidence in outputs [58], and mismatches between user expectations and tool outcomes [69].

In parallel, the user base of AI code generation tools has broadened far beyond professional developers. Non-developers, students, and individuals with little or no programming background can now produce functional code by describing goals in natural language [23, 39, 60], and practitioner

Authors' Contact Information: Ahmed Fawzy, a.fawzy@massey.ac.nz, School of Mathematical and Computational Sciences, Massey University, Palmerston North, New Zealand; Amjed Tahir, a.tahir@massey.ac.nz, School of Mathematical and Computational Sciences, Massey University, New Zealand; Kelly Blincoe, k.blincoe@auckland.ac.nz, Department of Electrical, Computer, and Software Engineering, University of Auckland, New Zealand.

accounts describe non-developers building applications for personal, organisational, and public-sector use [22, 77]. One industry report estimates that roughly a quarter of recent Y Combinator startups have codebases written almost entirely by AI [49]. This widening access creates a tension that prior work has begun to surface, but has not yet been examined across experience levels. End users without formal training often cannot reliably detect errors in AI-generated code even when explicitly warned to do so [72], and tend to rely on whether the code runs as a proxy for correctness [58, 69]. This widening user base has been accompanied by the emergence of new programming practices that differ from traditional AI-assisted programming.

One such practice is *vibe coding*, a programming practice in which users generate software through natural-language prompting, evaluate outputs primarily through execution, and iterate via reprompting rather than systematic debugging [21]. Unlike conventional AI-assisted programming, where developers retain deliberate control over planning and verification [5, 50], vibe coding emphasises outcome-driven 'trial-and-error' practice [21, 38]. Vibe coding lowers barriers to software creation by enabling users, including those with little or no programming experience, to build software through natural-language interaction with AI code generation tools, supporting rapid prototyping, learning, and creative exploration [21, 39]. However, this accessibility comes with a recurring speed–quality trade-off, as code inspection and structured quality assurance are often reduced [21]. A key concern is how users respond to this trade-off. Although only 33% of developers report trusting the accuracy of AI-generated code [66], reliance on these tools continues to grow, and user studies show that developers can articulate awareness of tool limitations without always translating this awareness into systematic verification practices [43, 73]. This creates a clear tension, as software becomes easier to produce but not necessarily to evaluate, debug, or secure.

Since different user groups engage in vibe coding practices, it is important to understand how different groups interact with it. Prior work has examined AI-assisted programming within specific user groups (i.e., professionals or students [43, 69]), or studied non-experts in isolation [23, 60]. More broadly, AI code generation has often been studied as a tool or workflow-centred phenomenon, with emphasis on productivity [6, 57, 82], usability [43, 69], code quality [45, 53, 67, 79], and interaction patterns [5, 50, 68], rather than as a coding practice shaped by users' programming backgrounds. As a result, it remains unclear how motivations, experiences, perceived code quality, and quality assurance (QA) practices differ across experience levels in vibe coding contexts. More importantly, it is also unclear whether recognising problems in AI-generated code leads to effective corrective action, or whether a gap exists between awareness and behaviour. This gap matters because if users share awareness of code quality issues but differ in how they respond to them, the risks of vibe coding depend not only on AI tools but also on users' capabilities. Characterising these differences can reveal where verification breaks down, identify which users are most vulnerable to relying on unverified AI-generated code, and inform the design of experience-aware tools, education in code evaluation and debugging, and organisational practices for safer adoption. To the best of our knowledge, no study has systematically compared vibe coding practices across non-developers, novice developers, and professional developers within a unified design.

To address this gap, we conduct a survey of those engaged in vibe coding practices, examining it as a cross-experience behavioural phenomenon. The study is grounded in themes identified by our prior grey literature review of practitioners' blogs, forums, and technical reports that characterised vibe coding along four behavioural dimensions: *motivations* (why users engage in the practice), *experiences* (what they encounter while doing so), *perceived code quality* (how they judge the resulting code), and *QA practices* (how they verify or validate it) [21]. Our previous review offered only a qualitative account synthesised from practitioner discourse, finding vibe coding was driven largely by speed and accessibility and accompanied by informal QA, but it did not measure how these behaviours varied across users with different programming backgrounds. We address this

by operationalising the four dimensions as quantitative, Likert-based constructs and examining how they differ across three user groups: non-developers, novice developers, and professional developers. We collected data from 181 participants, of which 162 met the data quality criteria and were evenly distributed across the three groups (54 each).

Our results reveal a consistent pattern that experience shapes some aspects of vibe coding behaviour but not others. Participants across all experience levels report similar experiences and perceptions of AI-generated code quality, indicating a shared awareness of both the benefits and limitations of vibe coding. In contrast, clear differences emerge in motivations, interaction styles, and QA practices. Non-developers are most motivated by accessibility and empowerment and more often delegate code generation entirely to the AI; novices are more motivated by learning and experimentation; and professionals engage in more iterative dialogue, provide richer contextual input, and use vibe coding more often in work-related contexts. The clearest divergence is in QA, with less experienced participants relying more on reprompting rather than manual debugging and reporting greater difficulty understanding AI-generated code when errors arise, with non-developers the only group reporting that they *never* check AI-generated code before use, whereas approximately 45% of professionals report *always* checking it.

These findings suggest a potential *perception–action gap*. All user groups share awareness of the AI-generated code limitations, but their ability to respond to them varies with experience. The risks of vibe coding are therefore shaped not only by the AI code generation tools, but also by differences in user capability across experience levels. This has direct implications for how vibe coding should be supported. AI code generation tools should provide stronger scaffolding for less experienced users (e.g., proactive error surfacing, explanatory support, and uncertainty signalling) while remaining unobtrusive for experts, and education should treat the evaluation of AI-generated code as a distinct skill alongside prompting.

2 Background and Related Work

2.1 AI-Assisted Programming and Code Generation

The rapid integration of AI code generation tools represents a significant transformation in software development practices [11, 82]. AI code generation and conversational AI assistant tools have become embedded in development workflows, supporting code generation, completion, and debugging tasks [37, 80, 81]. These tools leverage LLMs trained on large-scale code corpora to generate contextually relevant code from natural-language prompts, enabling rapid, iterative development cycles [11, 26, 74]. Recent empirical studies show that while AI code generation tools improve productivity for routine programming tasks, their performance varies depending on task complexity and context [53, 69]. Developers often use these tools to reduce keystroke effort, accelerate development, and recall syntax or APIs [43]. Interaction patterns extend beyond simple code completion; developers engage in iterative prompting and refinement workflows that shift attention from line-by-line coding to higher-level task specification [5]. However, even with these tools available, developers frequently refine or discard AI-generated suggestions. Recent studies have consistently report a gap between how useful developers expect AI tools to be and how effectively they integrate them into their actual workflows [43, 69], a pattern that persists even as AI code generation tools have grown considerably more capable [6, 12]. Notably, prior work has focused primarily on two groups: professional software developers and computer science students [5, 43, 53, 69, 81]. It remains unclear whether the productivity benefits, interaction patterns, and failure modes extend to users who lack formal programming training (i.e., non-developers) – a population that has grown substantially as AI code generation tools have become widely accessible [23, 64].

2.2 Experience and Human–AI Interaction in Programming

The integration of AI assistants into development workflows has introduced new patterns of human–AI interaction, shifting the developer's role from writing code directly to specifying, evaluating, and refining AI-generated outputs [5, 50]. This interaction model redistributes cognitive effort; rather than reasoning only about implementation, users increasingly need to assess and steer AI-generated code suggestions [43]. Observational research shows that this shift is also reflected in how developers direct their attention, with reviewers of AI-generated code tending to scan the high-level structure rather than closely examine implementation details [68].

This shift is particularly important because AI-assisted programming now involves users with varying levels of software development experience. Prior research on end-user programming has long shown that people without formal software engineering training create software artefacts to support their own goals [51, 52, 64]. Recent studies suggest that AI code generation tools extend this pattern by enabling non-expert users to produce functional code for practical tasks [23, 39]. However, these users may lack the conceptual knowledge needed to assess whether generated code is correct, robust, or maintainable [60], and may rely on surface-level indicators, such as whether the code runs without error, as a proxy for correctness [58, 69]. These concerns are especially relevant in AI-assisted contexts, where AI-generated code may appear correct while concealing deeper issues [17, 43].

Experience level also shapes how these patterns play out in practice. Novice developers benefit from reduced syntactic burden and faster feedback cycles, but are less able to critically evaluate the outputs they receive [57, 60]. Professional developers, despite being more capable of identifying problems, still exhibit reliance on AI assistance even for tasks within their expertise [8, 82]. Across experience levels, studies reported that developers recognise potential risks in AI-generated code yet do not consistently adjust their validation behaviour in response [58, 69, 73]. However, these findings are drawn mainly from separate studies targeting distinct populations, typically either professional developers or computing students, rather than from designs that directly compare non-developers, novices, and professionals [43, 60, 69].

2.3 Code Quality of AI-Generated Code

The quality of AI-generated code is a persistent concern [79]. While AI assistants can produce syntactically valid and functional code for straightforward tasks, empirical evaluations show that non-functional properties (such as security, maintainability, and correctness under edge cases) remain problematic [45, 53, 67]. Recent longitudinal evidence reinforces this concern at the project level. A difference-in-differences study of open-source repositories adopting an agentic AI coding assistant found that short-term gains in development velocity were accompanied by a substantial and persistent rise in code complexity and static analysis warnings, with the accumulating technical debt in turn associated with slower development over time [32]. Security risks have received particular attention. Previous research shown that AI-generated code contain security weaknesses, sometimes in large numbers [27, 55], and users tend to rate AI-generated code as more secure than it actually is [58]. Novice users are less able to critically evaluate AI-generated code and more likely to accept suggestions without understanding them [17, 60], suggesting that overconfidence about security may be particularly pronounced among less experienced users. Despite the practical importance of this possibility, no study has directly measured how the confidence–quality gap varies across user experience levels in an AI-assisted coding context. Quality assurance practices in AI-assisted development also differ from traditional workflows [43, 69]. Rather than applying systematic testing or structured code review [3], developers frequently rely on running the code to see whether it works, treating successful execution as sufficient validation [69]. Approaches

such as test-driven development can help constrain the scope of AI-generated code [20], but their adoption in AI-assisted workflows remains limited [20, 46].

3 Methodology

In this study, we collected data from practitioners through an online questionnaire. to investigate how *vibe coding* practices vary across coders with different levels of programming experience. The questionnaire was designed following established guidelines for survey research in software engineering. We drew on the principles for conducting online questionnaires in software engineering described by Punter et al. [61] and the structured questionnaire process proposed by Linåker et al. [44]. These guidelines informed the selection practices relevant to our study (e.g., survey design, pilot testing, recruitment, data quality, and transparent reporting of the study procedure). In the following, we explain the research questions, the study design, questionnaire development, participant recruitment, data quality procedures, and data analysis methods. We show our overall research workflow in Figure 1.

3.1 Research Questions

The study addresses the following research questions:

- **RQ1 (Motivation):** *How do motivations for engaging in vibe coding differ between non-, novice, and professional developers?*
  This question examines the reasons individuals adopt vibe coding. Potential motivations include increasing development speed, reducing cognitive effort, supporting learning, or enabling creative exploration. Comparing these motivations across experience levels helps determine whether vibe coding serves different purposes for individuals with different programming backgrounds.
- **RQ2 (Experience):** *How do experiences of vibe coding differ between non-, novice, and professional developers?*
  This research question focuses on participants' subjective experiences when engaging in vibe coding. These experiences include perceptions of success, frustration, flow, confidence, and reliance on iterative prompting. Examining these experiences across groups provides insight into how prior programming expertise shapes the practical use of AI-assisted coding.
- **RQ3 (Code Quality Perception):** *How do perceptions of AI-generated code quality produced through vibe coding differ between non-, novice, and professional developers?*
  This question investigates how participants evaluate the quality of AI-generated code. In particular, it considers perceptions of correctness, maintainability, fragility, and suitability for real-world or production use. Differences across experience levels help reveal how technical expertise influences trust in and evaluation of AI-generated code.
- **RQ4 (QA Practices):** *How do QA practices during vibe coding differ between non-, novice, and professional developers?*
  This research question examines how participants verify or validate the code generated by AI tools. It includes practices such as testing, code inspection, debugging, reprompting, or delegating verification tasks to the AI system. Comparing these behaviours across experience levels provides insight into how traditional software engineering QA practices are adapted, modified, or bypassed during vibe coding.

3.2 Questionnaire Design

This study employs a quantitative cross-sectional approach, in which data are collected from participants at a single point in time to enable systematic comparison between independent groups

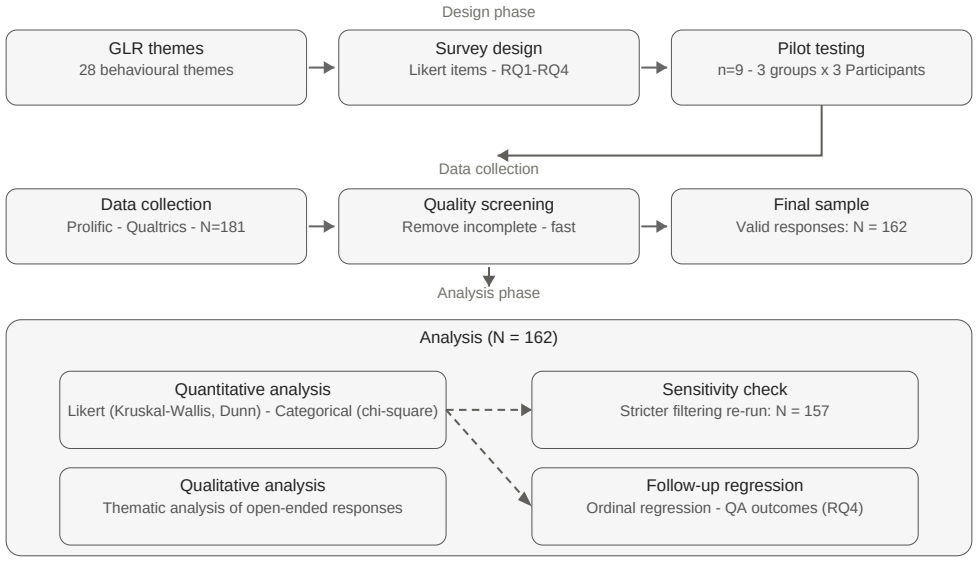

Fig. 1. An overview of the questionnaire workflow

[15]. Such designs are well-suited for examining group-level differences in motivations, experiences, perceptions of code quality, and QA practices associated with vibe coding (e.g., [3, 33]).

The questionnaire was informed by the findings of our recent GLR [21], which identified recurring behavioural themes describing how coders engage in vibe coding practices. These themes provided the conceptual foundation for defining the construct blocks examined in the questionnaire (see Table 1). Participants were grouped into three independent categories based on self-reported programming experience:

- **Non-developers:** Participants who reported that they do not have formal programming training or professional experience, but have used AI coding tools (e.g., ChatGPT, Copilot) to generate or modify code.
- **Novice developers:** Participants who reported having completed tutorials, coursework, or small personal projects involving programming or AI coding tools, but having limited real-world development experience.
- **Professional developers:** Participants who reported having professional or academic experience writing, maintaining, or reviewing code as part of their job, research, or formal development work.

3.3 Questionnaire Development

We used the themes identified in our recent GLR [21] to guide the development of the questionnaire. These themes were organised into four sections corresponding to the research questions: Motivation (RQ1), Experiences (RQ2), Code Quality Perception (RQ3), and QA Practices (RQ4). The questionnaire consisted of the following sections:

- **Consent and Definition:** This section introduced the concept of vibe coding and obtained the participants' consent.

- **Experience Classification and Vibe-Coding Style:** This section collected self-reported programming experience to classify participants into non-developer, novice developer, or professional developer groups. Additional items captured the context of vibe coding use (e.g., personal projects, work-related tasks, and learning activities), the AI code generation tools used (e.g., ChatGPT, GitHub Copilot, and Claude Code), the duration and frequency of vibe coding, and the types of projects undertaken (e.g., scripting, web/mobile applications, prototypes, etc.). A *Vibe-Coding Style* subsection was also included to capture participants' preferred workflow characteristics when interacting with AI code generation tools.
- **Vibe Coding Motivations (RQ1):** This section of the questionnaire assesses participants' motivation to engage in vibe coding. This includes reasons such as improving development speed, supporting learning, enhancing creativity, or reducing cognitive load. Participants rated statements such as "I use vibe coding to save time." on a five-point scale ranging from *strongly disagree* to *strongly agree*.
- **Participants' Experiences (RQ2):** This section captured participants' experiences while vibe coding, including feelings of success, frustration, flow, reprompting behaviour, and task abandonment. For example, participants rated statements such as "I often experience a quick flow where everything just works when vibe coding" and "I usually reprompt several times to get usable results."
- **Perceived Code Quality (RQ3):** This section examined participants' perceptions of the quality of AI-generated code, including perceived fragility, maintainability, and suitability for production use. For example, participants are asked to rate statements such as "Vibe-coded code is often fragile or error-prone" and "Vibe-coded code is good enough for demos but not for production use."
- **QA Practices (RQ4):** This section measured QA practices during vibe coding, including running code without checks, reviewing AI-generated code, re-prompting instead of manual debugging, and delegating QA tasks back to the AI. Both Likert-scale and frequency-based items were used to capture these behaviours. For example, participants rated statements such as "I run AI-generated code without checking it first" and reported how often they checked AI-generated code before use using response options ranging from *never* to *always*.
- **Demographics:** The final section collected demographic information from participants, including age range, gender, geographic location, and the highest level of education in a computing-related discipline. These variables were used for descriptive reporting and to characterise the participant population.
- **Standalone Open-Ended Questions:** We included several open-ended questions to gather broader reflections on AI-generated code features, suggestions for improvement, and general feedback on vibe coding practices. These items were presented to all participants and provided a qualitative context that complemented the structured Likert-scale responses.

Within each of these sections, every theme was operationalised as a Likert-scale item by simplifying its description (from the original study) into a measurable statement while preserving its original meaning, ensuring alignment between the qualitative synthesis and the quantitative measurement. For example, the theme *"Fast but Flawed"*, described in the GLR as "useful for quick tasks but unsuitable for production deployment," was operationalised as the item *"Code from vibe coding is useful for quick prototypes but often flawed"*. The full mapping between themes and questionnaire items is presented in Table 1, with detailed theme definitions reported in [21]. The questionnaire was implemented using Qualtrics[1]. Items were phrased as declarative statements and measured using five-point Likert-type scales (between 1 (Strongly Disagree) and 5 (Strongly Agree)),

[1] https://www.qualtrics.com/

enabling ordinal comparison across participants and supporting the planned non-parametric group comparison analyses [14, 24] (see Section 3.10.1). In addition to Likert-scale items, optional write-in fields were provided after each section to capture responses not covered by the predefined items and were analysed qualitatively as part of construct-coverage validation (see Section 3.4).

The *Vibe-Coding Style* items were informed by the development paradigm framework proposed in a recent survey of vibe coding models [30], which highlights the roles of human evaluation, structured development processes, and context management in human–AI coding interaction. The five style themes shown in Table 2 map onto these aspects, capturing variation in how coders delegate control to the AI, structure their interaction, and provide contextual guidance.

Table 1. Traceability mapping from themes identified in GLR [21] to items in questionnaire

Theme from GLR	**Questionnaire Item**	**RQ**
RQ1 – Motivations		
Speed & Efficiency	I vibe code mainly to save time.	RQ1
Accessibility & Empowerment	Vibe coding lets me build software I could not have built otherwise.	RQ1
Learning & Experimentation	I use vibe coding to learn or explore new languages or frameworks.	RQ1
Creative Exploration	I find vibe coding creative and enjoyable, and I often use it for exploration or play.	RQ1
Fast Prototyping	Vibe coding helps me quickly create prototypes or proofs of concept.	RQ1
Reducing Mental Effort	Vibe coding reduces the mental effort required for repetitive or boilerplate coding tasks.	RQ1
Frustration Avoidance	I use vibe coding to avoid tedious or frustrating debugging work.	RQ1
Escape from Complexity	Vibe coding lets me focus on ideas without worrying about complex design or architecture.	RQ1
Curiosity or Play	Sometimes I vibe-code just for fun, curiosity, or experimentation.	RQ1
RQ2 – Experiences		
Instant Success & Flow	I often experience a quick flow where everything just works when vibe coding.	RQ2
Prompt Struggle & Iteration	I usually reprompt several times to get usable results.	RQ2
Code Breakdown or Abandonment	I sometimes abandon a task when AI output becomes too messy or buggy.	RQ2
Fun & Creative Satisfaction	Working this way feels fun and creatively satisfying.	RQ2
AI Hallucinations	AI-generated code sometimes looks correct but behaves incorrectly when run.	RQ2

Theme from GLR	Questionnaire Item	RQ
Confusion or Misunderstanding	Sometimes I feel confused about what the AI understood from my prompt.	RQ2
RQ3 – Code Quality Perception		
Fast but Flawed	Code from vibe coding is useful for quick prototypes, but often flawed.	RQ3
Fragile or Error-Prone	Vibe-coded code is often fragile or error-prone.	RQ3
Sloppy or Low Maintainability	Vibe-coded code is hard to maintain or extend later.	RQ3
Prototype-Ready Only	Vibe-coded code is good enough for demos but not for production use.	RQ3
High Quality & Clean	AI-generated code is sometimes well-structured and clean.	RQ3
Misleading Confidence	AI-generated code looks correct but hides deeper problems.	RQ3
RQ4 – QA Practices		
Skipped QA	I run AI-generated code without checking it first.	RQ4
Manual Testing or Edits	I test or review AI-generated code carefully before using it.	RQ4
Uncritical Trust	I trust AI-generated code even when I don't fully understand it.	RQ4
Delegated QA to AI	I ask the AI to find or fix its own mistakes.	RQ4
Reprompting Instead of Debugging	If an error occurs, I reprompt instead of manually debugging.	RQ4
Run-and-See Validation	I usually run the code once to see if it works.	RQ4
QA Breakdown or Confusion	Sometimes I give up on debugging because I can't understand the AI's code.	RQ4

Table 2. Vibe-Coding Style

Style Theme	Questionnaire Item
AI-led Generation	I let the AI generate most or all of the code with little or no intervention.
Interactive Dialogue	I interact iteratively with the AI, prompting, reviewing, and refining code through dialogue.
Structured Planning	I outline or plan tasks before prompting the AI, using a structured or step-based approach.
Verification-Oriented Prompting	I define tests, checks, or verification steps before or alongside prompting the AI.
Rich Context Provision	I provide detailed context such as data, examples, or external files to guide the AI's coding.

Required and Optional Items. Core construct blocks (experience level classification and the main Likert-scale items corresponding to RQ1–RQ4) were configured as mandatory in Qualtrics to ensure complete data for between-group statistical comparisons. Standalone open-ended question, write-in fields following each Likert block, and demographic questions were optional to reduce participant burden and avoid requiring responses in non-essential fields.
Ethical approval. The survey was approved by the university ethics committee. Participation was voluntary and anonymous, and participants could ignore/leave out any optional question. The first author developed the initial draft of the questionnaire and refined it through repeated discussions with the other co-authors, resolving disagreements by consensus.

3.4 Construct Coverage Verification

To assess whether the questionnaire provided adequate thematic coverage, we examined the optional write-in responses associated with each Likert-scale construct block. These questions allowed participants to describe their vibe coding style, motivations, experiences, perceptions of AI-generated code, or quality-assurance (QA) practices in their own words when the predefined Likert statements did not fully reflect their perspectives. A total of 30 write-in responses were collected. Each response was systematically compared with the conceptual dimensions operationalised in the corresponding Likert-scale constructs. The responses were manually reviewed by the first author and checked by a co-author. Discrepancies were discussed and resolved through consensus, indicating that none of the responses introduced a new conceptual dimension beyond those represented in the questionnaire Likert items. Instead, many responses mapped directly to existing constructs. For example, one participant described their workflow as: "I usually work iteratively with the AI by describing what I want, reviewing the generated code, running it, and making small adjustments as needed", which corresponds to the Likert item: "I interact iteratively with the AI, prompting, reviewing, and refining code through dialogue". Other responses provided contextual descriptions consistent with the predefined themes. For instance, a participant noted that "AI-generated code can look clean and correct at first glance, but deeper issues sometimes appear when running or extending it", which aligns with the Likert item: "Vibe-coded code is hard to maintain or extend later". These observations provide additional evidence of construct coverage and support the survey instrument's content validity, suggesting that the questionnaire adequately captured the conceptual scope of vibe coding practices examined in this study.

3.5 Target Population

The target population consisted of vibe coders aged 18 years or older who had used AI code generation tools for vibe coding. This included non-developers with no formal programming background, novice developers with limited experience, and professional developers with professional or academic coding experience. Including all three groups enabled a systematic examination of how vibe coding practices vary with experience.

3.6 Determining Sample Size

We employed a power analysis to determine the minimum required sample size to detect differences in vibe coding practices across the three independent experience groups. We used a one-way between-groups ANOVA to estimate the minimum required sample size for group comparisons [13]. Although Likert-type responses are ordinal, they are commonly treated as approximately continuous in power analyses, thereby enabling ANOVA-based sample size estimation [24]. Furthermore, ANOVA-based power calculations tend to slightly overestimate the required sample size when the final analysis uses non-parametric tests, making this a conservative planning approach [14]. The power analysis assumed a medium effect size (Cohen's $f = 0.25$) across three groups (non-developers, novice developers, and professional developers), with a significance level of $\alpha = 0.05$ and desired power of 0.80, representing the smallest group difference the study was designed to detect.

The analysis was conducted in R using the "pwr" package (pwr.anova.test). Under these assumptions, the minimum required sample size was 159 participants (53 per group). A total of 181 responses were collected. After applying the predefined data-quality protocol (see Section 3.9), the analysed sample comprised 162 participants (54 per group), exceeding the minimum threshold.

To the best of our knowledge, no prior quantitative survey studies estimate effect sizes for differences in vibe coding practices across experience levels. Following standard guidance, a medium effect size (Cohen's $f = 0.25$) was assumed for planning purposes [13]. This assumption defines the smallest effect the study was designed to reliably detect and does not imply that the observed effects were expected to be of medium magnitude.

3.7 Pilot Study

To validate the clarity and suitability of the questionnaire, we conducted a pilot run before its full deployment. From the pool of participants (see Section 3.8) who reported engaging in vibe coding, we randomly selected 9 candidates (3 from each experience group) and invited them to participate in the pilot study. The pilot run collected feedback on whether the questionnaire questions were clear, well-worded, and appropriately captured the intended constructs (motivations, experiences, perceptions of AI-generated code, and QA practices). Participants were also invited to comment on potential issues, such as ambiguity, bias, or gaps.

Based on the pilot survey feedback, we refined the questionnaire by adjusting response scales, clarifying wording, and removing sources of bias. Using the revised version, we deployed the final questionnaire.

3.8 Data Collection

We used Prolific [2] to recruit human participants, with recruitment conducted in multiple stages. First, we created and published two screening questions on Prolific to identify potential participants who (1) had experience with vibe coding and (2) belonged to one of the three target groups: non-software developers, novice developers, or professional developers. This screening process resulted in a pool

[2] https://www.prolific.com/

of 665 candidate participants, consisting of 299 non-software developers, 228 novice developers, and 138 professional developers. Of these, 386 reported engaging in vibe coding, while 279 reported no prior experience with vibe coding. From the pool of eligible vibe coding participants, we invited 260 participants. We received responses from 181 participants. The experience-classification question was administered twice, at screening (5 January 2026) and again at the start of the main questionnaire (3 February 2026), approximately four weeks apart. The main questionnaire response was used for group assignment. Of the 162 participants in the sample, 96 could be matched to a screening response via their Prolific identifier. Among these, 71 (74.0%) gave the same classification on both occasions, while 25 (26.0%) reported a different one, with 17 of the changes drifting toward the middle "novice" category and only one participant moving between non-developer and professional developer. Given the four-week gap, we did not exclude participants whose responses differed.

3.9 Data Quality and Reliability Checks

Before conducting our analysis, the questionnaire responses were screened using a structured data quality protocol consistent with established recommendations for detecting inattentive responding (i.e., answering without carefully reading questionnaire items) and insufficient-effort responding (i.e., responses provided with minimal cognitive engagement) in online questionnaires [16, 42, 48]. All screening procedures were pre-specified and applied uniformly across experience groups to minimise the risk of systematic bias. The screening protocol was implemented in predefined stages. Steps 1–3 constituted the sequential exclusion criteria: responses failing a given criterion were removed before proceeding to the next stage. Steps 4 and 5 served as additional validation checks and did not independently result in participant exclusion.

***Step 1**: Completion filtering.* Only fully completed responses were retained for analysis. Responses that were started but not finished were excluded. In total, 13 incomplete responses were removed.

***Step 2**: Time-based cognitive plausibility.* We also examined completion times to detect implausibly rapid responses. The median completion time was 9.8 mins (M = 11.9, SD = 7.3 mins). Following best-practice guidance for detecting inattentive and insufficient-effort responses in online questionnaires [16, 18, 36, 42, 48], we derived a conservative lower-bound estimate of the minimal attentive completion time based on the questionnaire's structure. This estimate accounted for the time required to read the consent and introductory definition, respond to the 33 mandatory items, and provide minimal engagement with the demographic and optional open-ended questions. Under these assumptions, the minimum plausible attentive completion time was estimated at approximately 4 mins. Responses completed in under four mins were therefore considered inconsistent with attentive responding. The 4 mins cutoff corresponds to approximately 41% of the observed median completion time (9.8 mins) and falls below the empirical lower tail of the completion-time distribution (5th percentile $\approx$ 5 mins), indicating that it lies well below typical completion durations. Responses below this threshold were classified as implausibly rapid, resulting in the exclusion of 6 responses.

***Step 3**: Response-pattern diagnostics.* We applied response-pattern diagnostics across all Likert-scale items to detect straight-lining and invariant responding. Straight-lining refers to selecting the same response option for all Likert items. Invariant responding refers to zero variance (standard deviation = 0) across all Likert responses. Such patterns are commonly associated with satisficing behaviour, whereby respondents provide minimally effortful answers rather than evaluating each item carefully [16, 48]. No responses met these criteria; therefore, no additional exclusions were

applied at this stage.

***Step 4**: Open-ended engagement validation.* All open-ended responses were screened for substantive engagement (interpretable content directly related to the question prompt). For example, the question "What features would make AI-generated code (vibe coding output) easier and safer for you to use or trust?" received meaningful responses from all 162 participants. Empty or clearly non-substantive entries (e.g., random characters or irrelevant text) were identified and flagged in the dataset. This step served as an additional engagement validation layer and did not result in participant removal.

***Step 5**: Robustness verification.* As a robustness check, we repeated the full analysis using a stricter completion-time threshold of five minutes, yielding a reduced sample of N = 157. All inferential tests (Kruskal–Wallis with Benjamini–Hochberg correction, effect sizes, and Dunn's post-hoc comparisons) were re-run on this sample. The sensitivity analysis preserved the same pattern of statistically significant differences, effect-size magnitudes, and central tendencies across all four research questions: experience-level differences remained concentrated in interaction style, motivations (RQ1), and QA practices (RQ4), with no significant differences in experiences (RQ2) or perceived code quality (RQ3). No previously significant result disappeared, and no new significant result emerged, indicating that the study's conclusions are not sensitive to the response-time threshold.

Overall, 19 responses (10.5% of the original sample) were excluded during data screening, yielding a final analytical sample of 162 participants. These procedures were implemented to reduce measurement error arising from inattentive or insufficient-effort responding while preserving valid responses and minimising the risk of systematic exclusion bias.

3.10 Data Analysis

We analysed the survey responses using descriptive and inferential statistics. Descriptive statistics summarised response distributions across experience groups, while inferential tests examined whether statistically significant differences existed between groups. Most questionnaire items were measured on 5-point Likert scales and were therefore analysed using non-parametric statistical methods. Questions capturing categorical responses (e.g., application context and project types) were analysed using chi-square tests of independence.

3.10.1 Quantitative Statistical Analysis. **Descriptive Analysis.** Before conducting inferential statistical tests, we summarised survey responses using descriptive statistics for each experience group (non-developers, novices, and professionals). Because the questionnaire items were measured on a 5-point Likert scale, responses were treated as ordinal data. For each Likert item, the median and interquartile range (IQR) were calculated for each group. The IQR (the range between the 25th and 75th percentiles) reflects the dispersion of responses within the group. These statistics are appropriate for ordinal data because they do not assume equal distances between scale points [24]. Descriptive statistics were used to characterise overall patterns of agreement and illustrate how responses differed across experience groups before formal statistical testing. In particular, comparing medians across groups helps illustrate potential differences in central tendency prior to formal statistical testing.

Inferential Statistical Tests. For each questionnaire item, differences between the three independent experience groups (non-developers, novices, and professionals) were assessed using the Kruskal–Wallis H test [24]. The Kruskal–Wallis H test was selected because the questionnaire responses were measured on ordinal Likert-type scales and the study compares more than two

independent groups (non-developers, novices, and professionals). As a rank-based non-parametric test, it does not assume normally distributed data and is therefore appropriate for analysing ordinal and potentially skewed responses [14]. In addition to statistical significance testing, effect sizes were calculated using epsilon-squared (ε^2). This measure estimates the proportion of variability in ranked responses that can be attributed to differences between groups. Epsilon-squared was selected because it is well-suited to rank-based non-parametric tests and provides an interpretable estimate of effect magnitude for Kruskal-Wallis comparisons.

Multiple comparisons correction. We also carried out a significant correction for multiple testing. For example, RQ1 (motivations) consisted of several individual Likert-scale items, each analysed separately using the Kruskal-Wallis test. Conducting multiple statistical tests within the same research question increases the probability of identifying significant results purely by chance. To address this, p-values were adjusted using the Benjamini–Hochberg false discovery rate (FDR) procedure [7]. The Benjamini–Hochberg method controls the expected proportion of false discoveries among rejected hypotheses while maintaining higher statistical power than family-wise error rate corrections. FDR-based approaches are widely recommended in behavioural and empirical research because they are more likely to detect real differences (i.e., have higher statistical power) than more conservative corrections such as Bonferroni, which can miss real effects (Type II errors) [31, 71]. For questionnaire items that remained statistically significant after the Benjamini–Hochberg adjustment, Dunn's post-hoc pairwise comparisons were conducted to determine which specific experience groups differed. To control the increased risk of false-positive findings when performing multiple pairwise comparisons, Holm's correction was applied to the Dunn test p-values [34]. Holm's sequential procedure controls the family-wise error rate while being less conservative and more powerful than the traditional Bonferroni correction.

Categorical Comparisons. We analysed categorical questionnaire questions using inferential statistical tests appropriate for nominal data. Two questionnaire questions captured categorical variables rather than Likert-scale responses: (*Vibe-Coding Application Context* and *Types of Projects or Tasks Performed with Vibe Coding*). For these items, differences between experience groups were analysed using Pearson's chi-square (χ^2) test of independence. When significant associations were observed, effect sizes were reported using Cramér's V.

Regression Analysis. We conducted a follow-up analysis to examine which dimensions of experience drive differences in QA (RQ4). We modelled the three significant QA outcomes using ordinal logistic regression [1, 47], with experience group as the main predictor (non-developers as the reference category) and three usage-related controls: weekly vibe coding hours, adoption duration, and weekly independent (non-vibe) coding activity. These controls were entered as ordered numeric scores. The proportional-odds assumption was verified for all models using the Brant test [9]. Multicollinearity was assessed using generalised variance inflation factors (GVIF) [25].

3.10.2 Qualitative Analysis. We conducted a qualitative analysis on responses to the open-ended survey questions , which asked participants: *"What features would make AI-generated code (vibe coding output) easier and safer for you to use or trust?"* and providing any additional comments.

Responses were analysed using a lightweight thematic analysis [10] to identify recurring patterns in participants' descriptions. First, all responses were read in full to familiarise the researchers with the data. The first author then generated initial codes representing meaningful units of text (e.g., requests for better documentation, improved testing support, or stronger security guarantees). Related codes were subsequently grouped into broader themes that captured common expectations for improving the usability and trustworthiness of AI-generated code. Themes were iteratively reviewed and refined by the first author and discussed with other co-authors to ensure that coded responses were consistently interpreted and that the resulting themes accurately reflected the

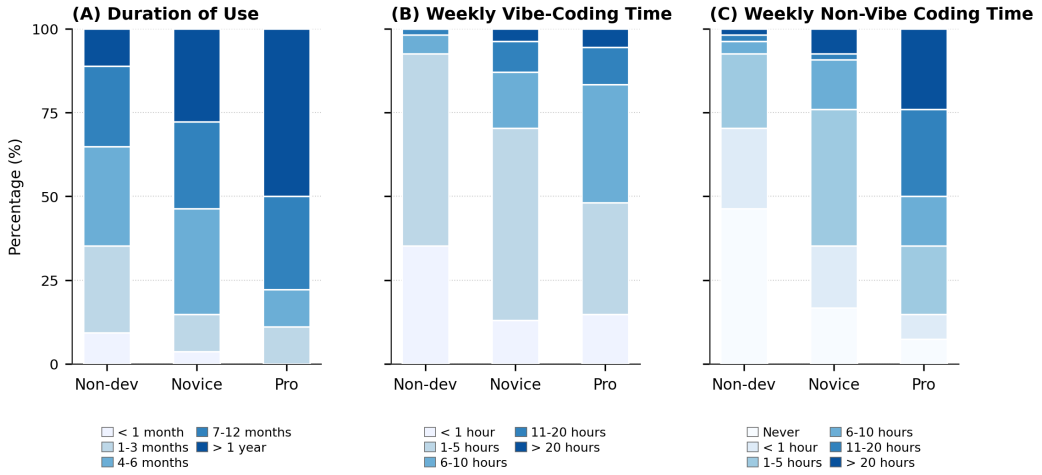

Fig. 2. Vibe coding engagement across experience groups.

dataset. The qualitative themes were then used to complement the quantitative findings by providing concrete examples of how participants described desired improvements to vibe coding outputs.

4 Results

4.1 Sample Overview

After applying the predefined data quality protocol (Section 3.9), the final analytical sample comprised 162 participants, evenly distributed across three independent experience groups: 54 non-software developers, novice developers, and experienced developers, each.

4.1.1 Participant Demographics. The three experience groups were broadly comparable in age but differed more in gender composition and educational background. Participants were predominantly aged 25–44 (n = 116, 71.6%), with 25–34 the largest category (n = 80, 49.4%). Most participants self-identified as men (n = 101, 62.3%) and women (n = 59, 36.4%). The sample reflects broad international participation across more than 20 countries, the largest groups being South Africa (n = 29, 17.9%), the United Kingdom and the United States (n = 16, 9.9% each), Portugal (n = 13, 8.0%), and Brazil (n = 12, 7.4%). Educational background varied across the sample: the most common response was a bachelor's or honours degree (n = 71, 43.8%), followed by a master's degree or higher (n = 24, 14.8%).

4.1.2 Vibe Coding Engagement. Participants reported sustained but heterogeneous engagement with vibe coding, and all three engagement measures differed significantly across experience groups (Figure 2). Adoption duration varied by experience ($\chi^2(8) = 30.06$, $p < .001$, $V = .305$): experienced developers were the most likely to report long-term use (50% reporting more than one year), whereas non-developers were more commonly recent adopters. Weekly vibe coding intensity showed a similar pattern ($\chi^2(8) = 30.99$, $p < .001$, $V = .309$), with non-developers concentrated in lower-usage categories and experienced developers reporting heavier weekly use. The biggest difference was in coding activity *without* vibe coding ($\chi^2(10) = 66.73$, $p < .001$, $V = .454$): non-developers most often reported never coding without vibe coding (46.3%), indicating high reliance on AI-assisted workflows, whereas roughly half of experienced developers reported 11 or more hours of independent coding per week. Full distributions are shown in Figure 2.

Table 3. Vibe coding context by experience groups (%)

Context	Non-dev	Novice	Pro
Work (job / freelance)	**27.8**	**31.5**	**68.5**
Personal Projects	55.6	72.2	59.3
Hobby / creative	**64.8**	**53.7**	**35.2**
Education	20.4	38.9	33.3

Table 4. Vibe coding task types by experience groups (%)

Task Type	Non-dev	Novice	Pro
Small scripts / automation	51.9	64.8	70.4
Creative / hobby projects	59.3	50.0	51.9
Data analysis / visualisation	31.5	44.4	44.4
Assignments / learning	**33.3**	**50.0**	**25.9**
Web / mobile apps	31.5	31.5	37.0
Work-related production	**16.7**	**14.8**	**33.3**
Prototypes / demos	**9.3**	**24.1**	**27.8**

4.1.3 Vibe Coding Application Context. Across the sample, vibe coding was most commonly used for personal projects (62.3%) and hobby or creative exploration (51.2%), followed by work-related use (42.6%) and education (30.9%) (Table 3). Two contexts differed significantly by experience: work-related use rose sharply with experience ($\chi^2(2) = 22.42$, $p < .001$, $V = .37$), reported by 68.5% of experienced developers but only 27.8% of non-developers, whereas hobby use showed the opposite trend ($\chi^2(2) = 9.69$, $p = .008$, $V = .25$), highest among non-developers (64.8%) and lowest among experienced developers (35.2%).

By task type, vibe coding was most frequently used for small scripts or automation (62.3%), creative projects (53.7%), and data analysis (40.1%) (Table 4). Three task types differed significantly by experience. Assignments and learning tasks were most common among novices ($\chi^2(2) = 7.09$, $p = .029$, $V = .21$), while work-related production tasks and prototypes or demos were more common among experienced developers ($\chi^2(2) = 6.63$, $p = .036$, $V = .20$ and $\chi^2(2) = 6.39$, $p = .041$, $V = .20$, respectively).In terms of tools, ChatGPT was the most widely used across all groups (75.3%), followed by Gemini (39.5%) and GitHub Copilot (24.1%) (Table 5). Usage rates were broadly similar across experience groups; the only significant difference was Claude ($\chi^2(2) = 9.99$, $p = .007$, $V = .25$), which was used more by experienced developers (27.8%) than by novices (14.8%) or non-developers (5.6%).

4.2 Vibe Coding Style

Beyond the four research questions, we also examined how participants operationalise vibe coding through their *interaction style* (see Section 3.3), a complementary dimension not tied to a specific research question. It is measured through five style themes (Table 2):

(1) *AI-led generation* (delegating code generation to the AI code generation tools), (2) *interactive dialogue* (iterative prompting and refinement), (3) *structured planning* (pre-planning tasks before prompting), (4) *verification-oriented prompting* (defining tests or checks), and (5) *rich context provision*

Table 5. AI coding assistants used by experience groups (%)

AI Assistant	Non-dev	Novice	Pro
ChatGPT	77.8	68.5	79.6
Gemini	35.2	40.7	42.6
GitHub Copilot	18.5	25.9	27.8
Claude	5.6	14.8	27.8
Cursor	1.9	13.0	7.4
DeepSeek	1.9	1.9	1.9
Grok	3.7	3.7	3.7

(supplying detailed input such as data or examples). Figure 3 shows the distribution of responses across experience groups, while Table 6 summarises medians and interquartile ranges. Overall, responses were centred around agreement (Mdn $\approx$ 4), indicating that these practices are commonly adopted across all groups. Statistically significant differences were identified for three themes: *AI-led Generation* ($p_{\text{BH}} = .004$, $\varepsilon^2 = .079$), *Interactive Dialogue* ($p_{\text{BH}} = .005$, $\varepsilon^2 = .067$), and *Rich Context Provision* ($p_{\text{BH}} = .015$, $\varepsilon^2 = .047$). As shown in Table 7, non-developers report higher agreement than both novices and professionals on letting the AI code generation tools generate most or all of the code with little or no intervention (***AI-led Generation***), while novices and professionals did not differ significantly from each other. For ***Interactive Dialogue*** (capturing iterative prompting, reviewing, and refining code through dialogue), professionals report higher agreement than non-developers ($p_{\text{adj}} = .001$), with no significant difference between novices and either group. For ***Rich Context Provision*** (reflecting the provision of detailed context such as data, examples, or external files to guide the AI code generation tools), non-developers report lower agreement than both novices and professionals (both $p_{\text{adj}} = .019$), with no significant difference between novices and professionals. Structured planning and verification-oriented prompting did not differ significantly across groups ($p_{\text{BH}} \geq .177$; Table 6), with negligible effect sizes ($\varepsilon^2 \leq .012$). Median responses were consistent across groups (Mdn $\approx$ 4), indicating that these practices are relatively stable regardless of experience level.

Overall Pattern. Experience level shapes how participants operationalise vibe coding, with differences concentrated in three dimensions. Non-developers exhibit greater reliance on AI-led generation, reflecting a more passive engagement with the AI code generation tools. In contrast, professionals engage more actively through iterative dialogue and the provision of richer contextual input. Structured planning and verification-oriented prompting remain stable across groups, suggesting that these aspects of vibe coding behaviour are broadly shared regardless of experience. Together, these results indicate a shift from passive to more interactive and context-rich engagement as experience increases, reflecting differences in how participants guide, refine, and control AI-generated code.

4.3 Motivations for Vibe Coding (RQ1)

RQ1 examined participants' motivations for using vibe coding across nine themes. Figure 4 shows the distribution of responses across experience groups, while Table 8 summarises medians and interquartile ranges. Overall, motivations were consistently rated around agreement (Mdn $\approx$ 4) across all groups. Statistically significant differences were observed in two themes: *Accessibility & Empowerment* ($p_{\text{BH}} < .001$, $\varepsilon^2 = .110$) and *Learning & Experimentation* ($p_{\text{BH}} = .015$, $\varepsilon^2 = .059$). Table 7 shows that both non-developers and novices report higher agreement than professionals

Table 6. Vibe Coding Style across experience groups

Theme	Non-dev	Novice	Pro	p_{BH}	ε^2
AI-led Generation	**4 (1)**	**3 (2)**	**3 (1)**	**.004**	**.079**
Interactive Dialogue	**4 (2)**	**4 (1)**	**5 (1)**	**.005**	**.067**
Structured Planning	4 (1)	4 (2)	4 (1.75)	.177	.012
Verification-Oriented Prompting	4 (2)	4 (1)	4 (1)	.444	.000
Rich Context Provision	**3 (2)**	**4 (2)**	**4 (2)**	**.015**	**.047**

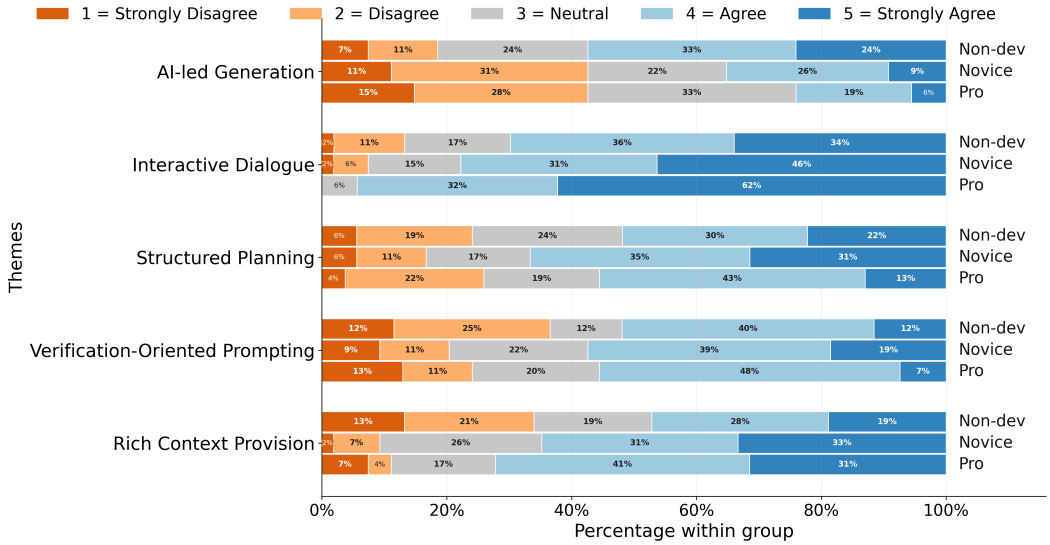

Fig. 3. Distribution of vibe coding style responses across groups.

Table 7. Pairwise comparisons for all statistically significant themes.

Section	Theme	Non-dev vs Novice	Non-dev vs Pro	Novice vs Pro
Vibe Coding Style	AI-led Generation	**.009** (Non-dev > Novice)	< **.001** (Non-dev > Pro)	Not significant
	Interactive Dialogue	Not significant	**.001** (Pro > Non-dev)	Not significant
	Rich Context Provision	**.019** (Novice > Non-dev)	**.019** (Pro > Non-dev)	Not significant
Motivations (RQ1)	Accessibility & Empowerment	Not significant	< **.001** (Non-dev > Pro)	**.004** (Novice > Pro)
	Learning & Experimentation	**.002** (Novice > Non-dev)	Not significant	Not significant
QA Practices (RQ4)	Reprompting Instead of Debugging	Not significant	**.008** (Non-dev > Pro)	Not significant
	QA Breakdown or Confusion	**.027** (Non-dev > Novice)	**.002** (Non-dev > Pro)	Not significant

on the item stating that vibe coding lets them build software they could not have built otherwise (***Accessibility & Empowerment***), with no significant difference between non-developers and novices. Furthermore, non-developers report lower agreement than novices on using vibe coding to learn or explore new languages or frameworks (***Learning & Experimentation***), with no other

Table 8. Motivation themes for vibe coding across experience groups.

Theme	Non-dev	Novice	Pro	p_{BH}	ε^2
Speed & Efficiency	4 (2)	4 (2)	4 (1)	.884	.000
Accessibility & Empowerment	**5 (1)**	**4 (2)**	**3 (3)**	**< .001**	**.110**
Learning & Experimentation	**3 (2)**	**4 (2)**	**4 (1.75)**	**.015**	**.059**
Creative Exploration	4 (1.75)	4 (2)	4 (2)	.884	.000
Fast Prototyping	4 (1.25)	5 (1)	5 (1)	.104	.018
Reducing Mental Effort	4 (1)	5 (1)	4 (1)	.177	.012
Frustration Avoidance	4 (2)	4 (2)	4 (2)	.884	.000
Escape from Complexity	4 (1)	4 (1)	4 (2)	.884	.000
Curiosity or Play	4 (2)	4 (2)	4 (2)	.884	.000

significant pairwise differences. All remaining motivation themes did not differ significantly across experience groups ($p_{\text{BH}} \geq .104$). Median responses were consistently around agreement (Mdn ≈ 4), indicating that motivations such as speed and efficiency, creative exploration, reduced mental effort, frustration avoidance, and curiosity are broadly shared across experience levels.

Overall Pattern. Motivations for vibe coding are broadly similar across experience levels, with differences limited to specific themes. Accessibility and empowerment are most strongly associated with non-developers, reflecting their role as an enabling mechanism, whereas learning and experimentation are more prominent among novices, indicating that they use vibe coding to develop programming understanding and experiment with code behaviour. In contrast, motivations related to speed, creativity, and reduced effort are consistently reported across groups, indicating that these motivations are largely independent of prior programming experience.

4.4 Experience with Vibe Coding (RQ2)

RQ2 examined participants' experiences during vibe coding, including both positive states (e.g., flow and creative satisfaction) and process-related challenges (e.g., iteration, hallucinations, and confusion). Figure 5 shows the distribution of responses across experience groups, while Table 9 summarises medians and interquartile ranges. No statistically significant differences were observed across experience levels for any theme ($p_{\text{BH}} \geq .437$, $\varepsilon^2 \leq .020$), and responses were broadly consistent across groups, ranging between neutral and agreement (Mdn = 3–4). Positive experiences were similarly reported across all groups. Participants moderately agreed that vibe coding is fun and creatively satisfying, and that it occasionally produces a quick flow where everything just works. Challenges were equally shared. Participants consistently agreed that reprompting is a common part of the process, that AI-generated code can appear correct but behave incorrectly when executed, and that they sometimes feel uncertain about what the AI code generation tools understood from their prompt. Task abandonment when AI-generated code becomes too messy or buggy was also reported at moderate levels across all groups.

Overall Pattern. Experiences with vibe coding are broadly similar across experience levels. Both positive experiences and challenges are reported at comparable levels regardless of prior programming experience. This contrasts with the motivational differences observed in RQ1, suggesting that once participants engage with vibe coding, the experience itself converges across experience levels.

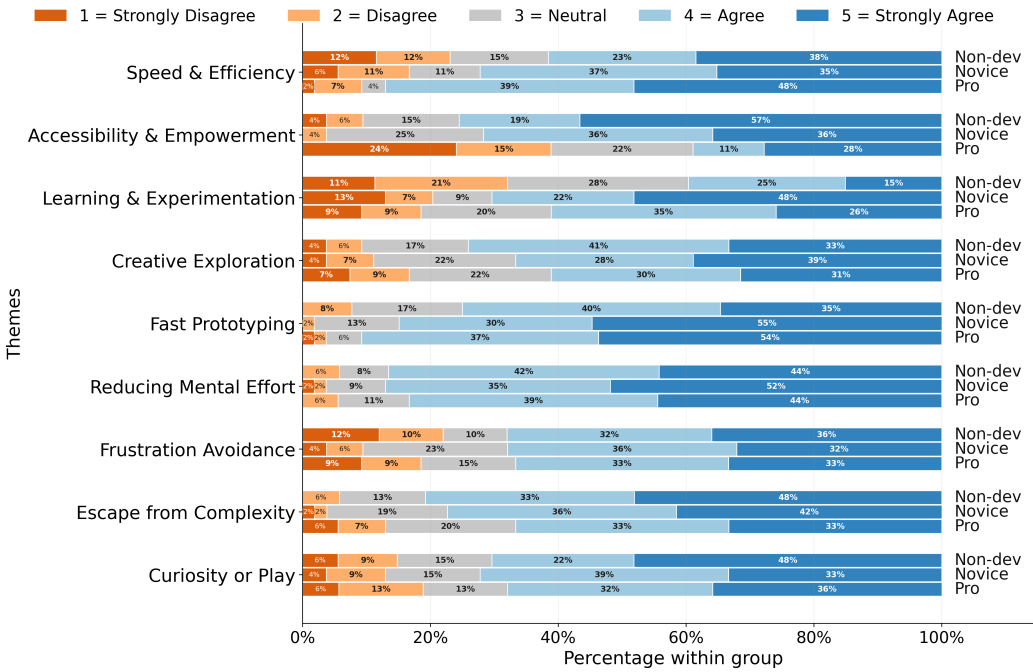

Fig. 4. Distribution of motivation responses across experience groups.

Table 9. Experience with vibe coding across experience groups.

Theme	Non-dev	Novice	Pro	p_{BH}	ε^2
Instant Success & Flow	3 (1)	3 (1)	3 (2)	.892	.000
Prompt Struggle & Iteration	4 (2)	4 (1.75)	4 (1)	.892	.000
Code Breakdown or Abandonment	4 (2.25)	3 (2)	3.5 (3)	.892	.000
Fun & Creative Satisfaction	4 (2)	4 (2)	4 (1)	.437	.020
AI Hallucinations	4 (1)	4 (2)	4 (1)	.437	.012
Confusion or Misunderstanding	4 (2)	3 (1)	4 (2)	.892	.000

4.5 Perceived Code Quality of the AI-Generated Code (RQ3)

RQ3 examined participants' perceptions of the quality, robustness, and production readiness of AI-generated code produced through vibe coding. Figure 6 shows the distribution of responses across experience groups, while Table 10 summarises medians and interquartile ranges. No statistically significant differences were observed across experience levels for any theme ($p_{\mathrm{BH}} = .884$ for all, $\varepsilon^2 \leq .008$), and responses were broadly consistent across groups, ranging between neutral and agreement (Mdn = 3–4). Participants held a cautiously mixed view of AI-generated code quality. On the positive side, participants moderately agreed that AI-generated code can sometimes be well-structured and clean. At the same time, participants consistently acknowledged its limitations: AI-generated code is perceived as useful for rapid prototyping but often flawed, fragile or error-prone, and more suitable for demos than production deployment. Participants also indicated

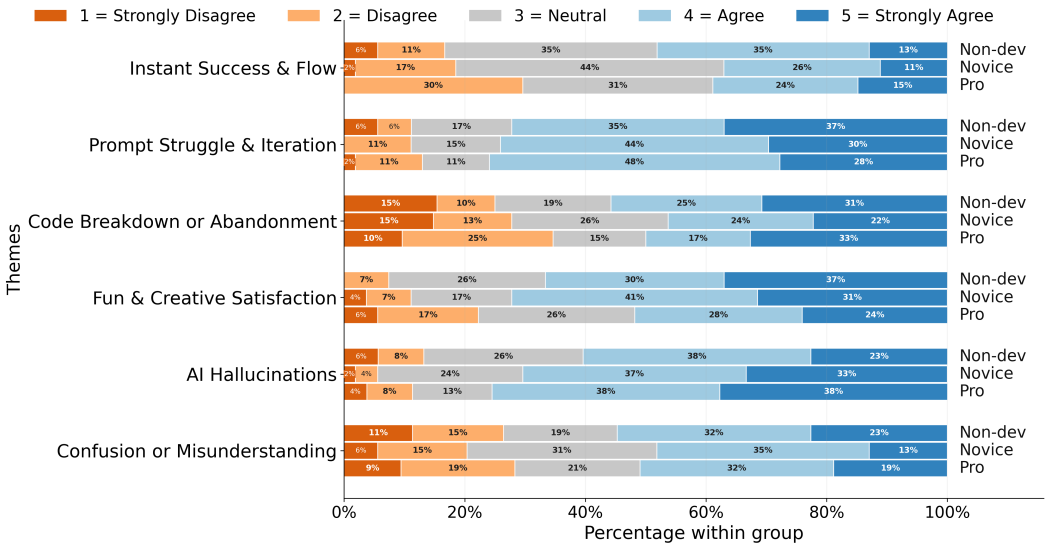

Fig. 5. Distribution of experience responses across groups.

Table 10. Perceived code quality of vibe-coded outputs (RQ3) across experience groups.

Theme	Non-dev	Novice	Pro	p_{BH}	ε^2
Fast but Flawed	4 (2)	4 (1)	4 (2)	.884	.008
Fragile or Error-Prone	4 (1)	3 (1)	4 (1)	.884	.000
Sloppy or Low Maintainability	3 (2)	3 (2)	3 (2)	.884	.000
Prototype-Ready Only	4 (1)	3.5 (1)	3 (2)	.884	.000
High Quality & Clean	4 (1)	4 (1)	4 (1)	.884	.000
Misleading Confidence	4 (1)	3 (1)	3.5 (1.75)	.884	.000

that AI-generated code may appear correct while concealing deeper issues, and reported neutral perceptions regarding its maintainability and structural quality.

Overall Pattern. Perceived code quality is broadly similar across experience levels. Participants consistently recognise both the strengths and limitations of AI-generated code, indicating that these perceptions are largely independent of prior programming experience. Notably, a general awareness of quality issues is shared across all groups, even though the ability to evaluate and act on those issues appears to vary with experience, as reflected in the QA practice differences observed in RQ4.

4.6 QA Practices when Vibe Coding (RQ4)

RQ4 examined how participants validate, test, and troubleshoot AI-generated code during vibe coding. Figure 7 shows the distribution of responses across experience groups, while Table 12 summarises medians and interquartile ranges. In contrast to RQ2 and RQ3, statistically significant differences across experience levels were observed for selected QA practices. Two themes showed statistically significant differences: *Reprompting Instead of Debugging* ($p_{\text{BH}} = .037$, $\varepsilon^2 = .045$) and

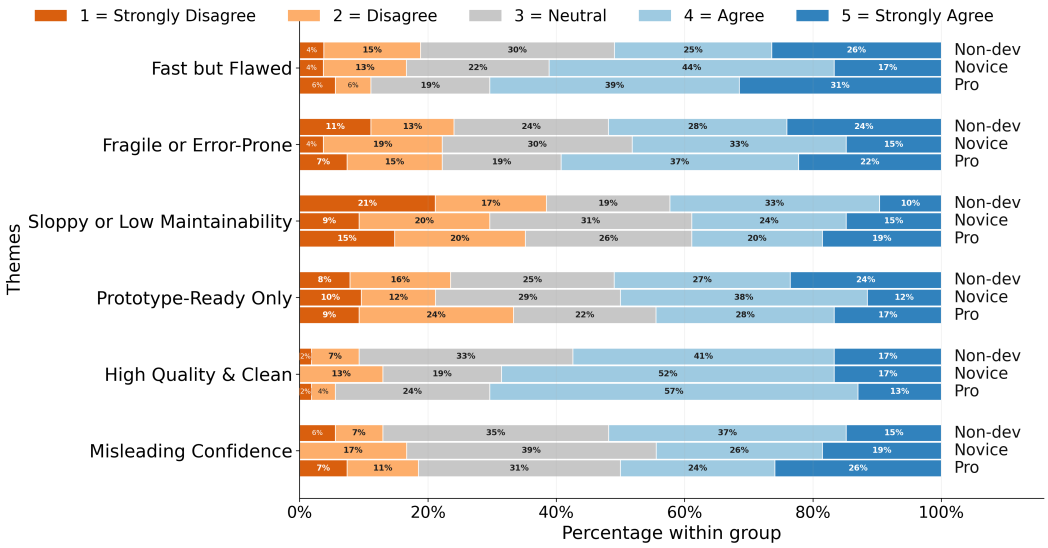

Fig. 6. Distribution of perceived code quality responses across groups.

QA Breakdown or Confusion ($p_{\text{BH}} = .012$, $\varepsilon^2 = .068$). As shown in Table 7, non-developers report higher reliance on reprompting rather than manually debugging errors (***Reprompting Instead of Debugging***) than professionals ($p_{\text{adj}} = .008$), while novices did not differ significantly from either group. For ***QA Breakdown or Confusion*** (capturing the extent to which participants give up on debugging because they cannot understand the AI-generated code), Non-developers report higher agreement than both novices ($p_{\text{adj}} = .027$) and professionals ($p_{\text{adj}} = .002$), with no significant difference between novices and professionals. All remaining QA practices (Skipped QA, Manual Testing or Edits, Uncritical Trust, Delegated QA to AI, and Run-and-See Validation) did not differ significantly across groups ($p_{\text{BH}} \geq .259$, $\varepsilon^2 \leq .015$), and manual testing and editing was commonly reported across all groups (Mdn = 4).

Frequency of Checking AI-Generated Code. As shown in Figure 8, checking frequency varied across experience groups ($\chi^2(8) = 16.04$, $p = .042$; Kruskal–Wallis $p = .013$). Professionals reported the highest rates of consistent checking, with approximately 45% reporting that they always check AI-generated code before using it. Novices were most concentrated in the *often* category (≈45%), while non-developers were more evenly distributed across lower-frequency categories and were the only group to report never checking AI-generated code prior to use.

Regression Analysis. To examine whether the QA differences observed above are explained by experience-group membership itself or by the underlying dimensions of experience (usage), we ran three ordinal logistic regressions, one per significant QA outcome (see Section 3.10.1). The three QA regression models (Table 11) all satisfied the proportional-odds assumption (Brant test, all $p > .78$), with negligible multicollinearity (all GVIF < 1.3). Across all three models, experience group did not significantly predict QA behaviour once the three usage dimensions of experience (weekly vibe coding hours, adoption duration, and weekly independent non-vibe coding activity) were included. Instead, the significant predictors were the usage dimensions themselves. Greater adoption duration and greater independent coding activity were each associated with less reprompting, and greater independent coding activity (coding without vibe coding) was also associated with more frequent checking of AI-generated code. This indicates that the QA differences are not arbitrary group

Table 11. Ordinal logistic regression of QA outcomes on experience group and usage variables. Cells report odds ratios (95% CI). Non-developers are the reference group.

Predictor	Reprompting	QA Breakdown	Checking Freq.
Novice (vs non-dev)	1.00 (0.65–1.54)	0.75 (0.49–1.14)	1.25 (0.81–1.93)
Professional (vs non-dev)	1.01 (0.61–1.68)	0.76 (0.46–1.26)	1.40 (0.83–2.34)
Weekly vibe hours	0.84 (0.69–1.03)	0.85 (0.70–1.03)	1.02 (0.84–1.25)
Adoption duration	**0.84 (0.72–0.99)**	0.89 (0.76–1.03)	1.01 (0.86–1.18)
Non-vibe coding hours	**0.85 (0.75–0.97)**	0.92 (0.81–1.04)	**1.15 (1.01–1.31)**

Bold = $p < .05$.

Table 12. QA practices during vibe coding (RQ4) across experience groups.

Theme	Non-dev	Novice	Pro	p_{BH}	ε^2
Skipped QA	2 (2.75)	2 (2)	2 (2)	.485	.000
Manual Testing or Edits	4 (1.75)	4 (1)	4 (2)	.259	.015
Uncritical Trust	3.5 (2)	3 (2)	3 (2)	.485	.000
Delegated QA to AI	4 (1)	4 (1)	4 (2)	.485	.000
Reprompting Instead of Debugging	**4 (2)**	**4 (1)**	**3 (2)**	**.037**	**.045**
Run-and-See Validation	4 (1)	4 (1)	4 (1)	.636	.000
QA Breakdown or Confusion	**4 (2)**	**3 (2)**	**2 (2.75)**	**.012**	**.068**

effects but are shaped by accumulated coding practice and tool exposure, the specific components of experience that distinguish the three groups.

Overall Pattern. QA practices show selective differences across experience levels. Differences emerge primarily in debugging-related behaviours, where less experienced participants report greater reliance on reprompting and higher levels of confusion when faced with AI-generated code they cannot understand. More experienced participants report lower levels on both measures and check AI-generated code more frequently and consistently. The regression analysis further showed that these differences are driven by accumulated coding practice and tool exposure rather than by experience-group category alone. Together, these results indicate that while perceptions of code quality are broadly shared, the ability to evaluate and respond to quality issues remains dependent on experience, specifically on accumulated coding practice and tool exposure.

4.7 Improving Trust in Code Generated by Vibe Coding

To complement our analysis, we analysed responses to an open-ended question asking participants what features would make AI-generated code (vibe coding output) easier and safer to use or trust. We received responses from all 162 participants. We conducted a thematic analysis of all responses and grouped all responses into four recurring themes (responses could be assigned to multiple themes). The most prominent theme was *reliability and correctness*. Participants emphasised the need for dependable outputs, early error detection, and functionally correct behaviour, requesting features such as "clear explanations of what the code does, automatic error or bug warnings, and easy suggestions for improvements or fixes." A second theme, *workflow support and control*, reflected a need for process-level visibility during development, specifically the ability to observe

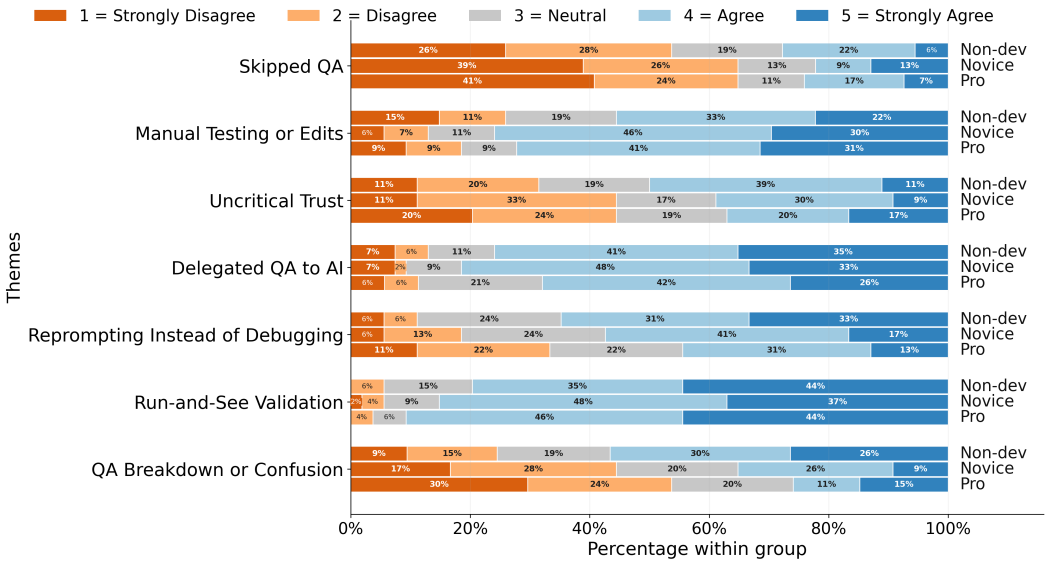

Fig. 7. Distribution of QA practice responses across experience groups.

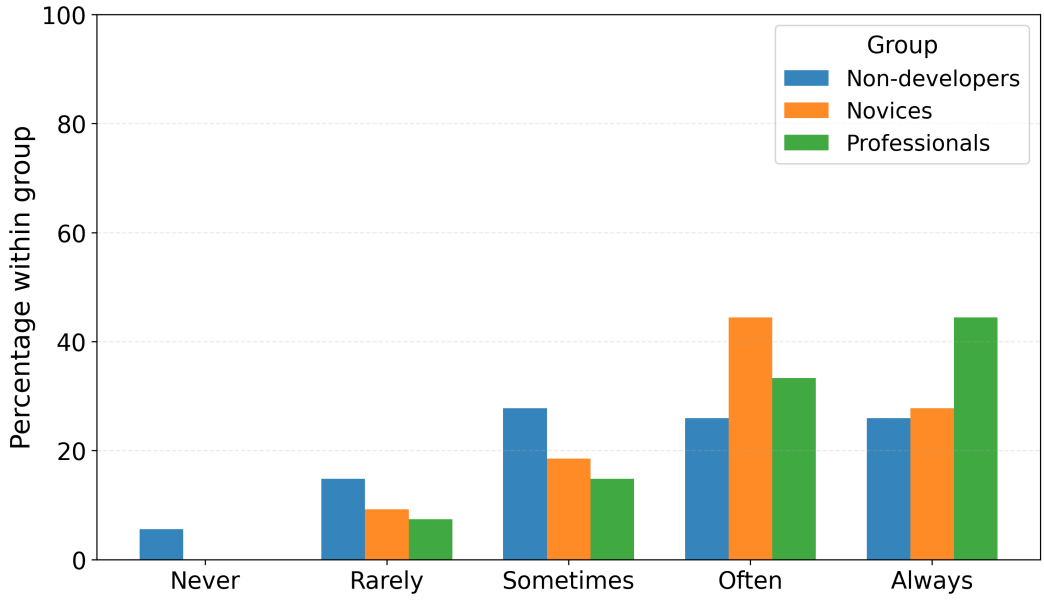

Fig. 8. Frequency of checking AI-generated code across experience groups.

how generated code executes and to receive step-by-step guidance as the tool produces it, with one participant wanting tools that "give reasons for the code as we go." A third theme, *transparency and understandability*, focused on comprehension of the code artifact itself: participants wanted clearer documentation and explanations of what the generated code does and why specific implementation

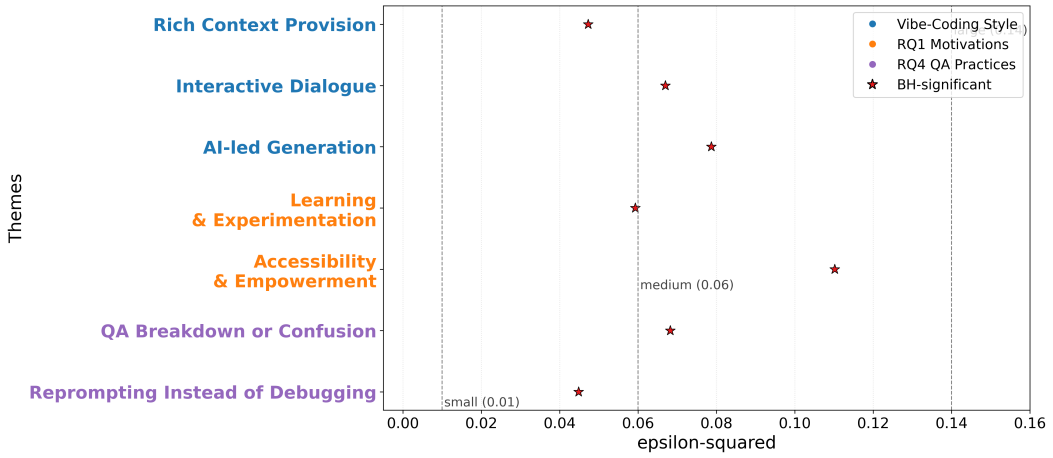

Fig. 9. Experience Influences Behaviour but Not Perception in Vibe Coding: Effect Sizes Across RQ1–RQ4 and Interaction Style

choices were made. The least common theme, *safety, privacy, and risk control*, reflected concerns about overly complex or risky outputs, with participants preferring tools that "do not suggest further and complicated output." These feature expectations were broadly consistent across the three experience groups, suggesting they are largely shared regardless of prior programming experience.

4.8 Summary of the Results

The results show that experience level does not uniformly shape vibe coding practices; instead, its influence is concentrated in specific parts of the vibe coding process. Differences are most evident in how participants *approach* and *handle failures* when vibe coding. Non-developers rely more on AI-led code generation and view it as an enabling tool. Novices emphasise learning from the code generation and view it as an enabling tool for a better software development experience. Professionals engage with vibe coding more through interactive prompting and structured guidance, yet perceive the experience and quality of AI-generated code in ways broadly similar to those of less-experienced groups. Across groups, participants recognise a similar balance of strengths (e.g., usefulness for rapid prototyping and well-structured output) and limitations (e.g., fragility, hidden bugs, and limited production readiness). The clearest differences emerge in quality assurance and debugging. Less experienced participants rely more on reprompting and report greater confusion, whereas more experienced participants report less reprompting, less debugging breakdown, and more frequent assessments of the AI-generated code. This suggests that experience level largely influences how participants respond to errors and limitations in AI-generated code, rather than how they perceive them.The regression analysis showed that these QA differences are driven by accumulated coding practice and tool exposure rather than by experience-group category alone.

The results also show varying expectations across the three groups, with participants consistently emphasising reliability, transparency, workflow support, and safety.

5 Discussion

5.1 Key Observations

Selective Influence of Experience: Across all research questions, a consistent pattern emerges: experience primarily influences behavioural themes, while reported experiences and perceived code quality remain largely stable. As shown in Figure 9, effect sizes are negligible for *experiences* (RQ2) and *perceived code quality* (RQ3), but small-to-moderate for *motivations* (RQ1), interaction style, and selected *QA practices* (RQ4).

This pattern is also evident in vibe coding interaction styles, where experience shapes how users engage with AI code generation tools, shifting from passive reliance on these tools, towards more iterative and context-rich interaction, while core practices such as planning and verification remain stable. In turn, awareness of risks in AI-generated code, such as fragility, hidden defects and bugs, and misleading correctness, is broadly shared at a general level between all three user groups. However, the ability to respond to these risks varies with experience. We interpret this pattern as a potential *perception–action gap*: an apparent decoupling between recognising potential issues and effectively addressing them. Because our measures are based on self-report rather than observed behaviour, we present this as an interpretation consistent with our data and a hypothesis for future behavioural studies to test, rather than a directly validated finding (Section 6). This observation aligns with prior findings showing that developers often rely on AI code generation tools despite recognising its limitations [69, 73, 78].

Vibe Coding as a Partially Democratising Practice: The absence of significant differences in RQ2 (*experiences*) and RQ3 (*perceived code quality*) is a notable result. Participants across all experience levels report similar interaction patterns, including flow, iterative prompting, and general awareness of code limitations. This convergence suggests that participants across all experience levels report broadly similar experiences when vibe coding, in which flow, iterative reprompting, awareness of hallucinations, and creative satisfaction emerge at comparable levels regardless of prior programming background, consistent with the goals of end-user programming to lower barriers to software creation [40, 64].

However, this convergence operates primarily at the level of perception rather than action. While access to code generation and high-level awareness of risks appears broadly distributed, the ability to evaluate, debug, and validate outputs remains uneven across experience levels. This indicates a form of *partial democratisation*, i.e., rather than eliminating barriers, vibe coding shifts them from code production to other barriers such as decomposing problems, specifying intent clearly, and valuating the correctness and robustness of AI-generated code.

Alternative explanations should also be considered. Similar perceptions of code quality may reflect shared exposure to discourse about AI limitations, rather than equivalent evaluative capability. Relatedly, our items measured awareness of code limitations at a general level (e.g., whether AI-generated code is error-prone or fragile) rather than probing specific risk categories such as security vulnerabilities. Similar responses to these high-level items therefore indicate broadly shared general awareness, but not necessarily an equivalent depth or accuracy of risk understanding across experience groups.

Likewise, behaviours such as reprompting may represent an efficient interaction strategy rather than a deficit. Nevertheless, the observed divergence in QA practices suggests that, even when risks are recognised, the capacity to act on this awareness remains experience-dependent.

Divergent Entry Points: Motivation and Interaction Style: Differences in specific motivations and interaction styles reflect distinct modes of engagement shaped by experience. Non-developers

are primarily driven by accessibility and tend to rely on AI-code generation with minimal structure. Novices emphasise learning and experimentation, using vibe coding as an exploratory environment. In contrast, professional developers are more goal-oriented, engaging in iterative dialogue and providing richer contextual input.

These differences suggest that vibe coding is not a uniform practice but a flexible interaction paradigm that adapts to coder intent and prior experience. Our findings regarding vibe coding application context (Section 4.1.3) further support this interpretation. Non-developers reported the highest levels of hobby-oriented use, suggesting that vibe coding serves as an accessible entry point for creative exploration for this group. In contrast, our survey results showed that novices engage more in goal-directed contexts such as learning and assignments, whereas experienced developers predominantly use it for professional tasks. This pattern indicates that differences in experience are reflected not only in behaviour, but also in how and why vibe coding is adopted.

Divergence in Quality Assurance Behaviour: The perception–action gap is most evident in QA practices. Less-experienced participants rely more on reprompting and report greater difficulty debugging, whereas experienced developers more consistently verify and evaluate outputs. This divergence occurs despite similar perceptions of code quality, reinforcing that recognising potential issues does not necessarily translate into effective corrective action. A follow-up regression analysis reinforced this view: once accumulated coding practice and tool exposure were modelled directly, experience-group membership no longer carried independent predictive power. This indicates that QA behaviour is shaped by the specific dimensions of experience (coding practice and exposure) rather than by group category alone.

This pattern is consistent with prior work showing that debugging and code comprehension depend on experience, mental models, and the use of debugging and comprehension strategies [4, 41, 54, 65]. In the context of AI-assisted development, these capabilities remain critical but are no longer directly reflected in perceived code quality.

5.2 Implications

We discuss below several implications for vibe coding in research and practice.

Decoupling Perception and Action in Software Engineering Expertise: Prior work on expertise suggests that the ability to recognise and resolve problems develops together through experience and practice [19]. Our findings refine this view in the context of AI-assisted development. While general awareness of potential issues appears broadly accessible across experience levels, the ability to evaluate, debug, and verify code remains strongly experience-dependent. This indicates that AI-assisted development reshapes how expertise is expressed: perceptual awareness becomes more widely distributed, whereas action-oriented capabilities continue to differentiate levels of expertise.

Future research can distinguish between awareness-level and resolution-level competences when studying expertise in AI-assisted contexts, and examine how tools and software training can better support the transition between them.

Implications for AI-Coding Tools: The perception–action gap highlights a limitation of current AI code generation tools, which primarily support code generation but provide limited support for evaluation and verification. Addressing this gap requires those tools to actively support users in translating awareness into effective action, particularly for less experienced coders.

Building on these findings, we derive three design directions from both the observed QA behaviours (RQ4) and participants' reported needs for tool support. First, *proactive error surfacing*,

where tools identify and communicate potential issues at generation time in ways that guide concrete corrective steps. Second, *explanatory support*, providing context-sensitive explanations that help coders understand not only what the code does, but how to evaluate and adapt it. Third, *uncertainty signalling*, explicitly communicating assumptions and limitations to prompt verification behaviours rather than passive acceptance. These capabilities should be adaptive to coder experience, providing stronger scaffolding for less experienced coders while remaining unobtrusive for experts, in line with prior work on scaffolding and intelligent tutoring systems [62, 70, 75, 76].

Implications for Practice and Education: The perception–action gap suggests that adopting vibe coding without structured QA processes introduces risk, particularly in mixed-experience teams [58, 69]. Even when coders recognise limitations in AI-generated code, they may lack the skills required to act on this awareness effectively. Establishing explicit verification practices, including validation steps and human code review, is therefore essential [3].

For education, these findings highlight the need to treat evaluation of AI-generated code as a distinct skill. Prior work has shown that AI-generated code can contain security vulnerabilities and functional defects, including subtle bugs that may not be immediately visible during execution [2, 27, 29, 55, 67]. Instruction should extend beyond prompting to include testing outputs, identifying edge cases, challenging assumptions, and reasoning about code behaviour. Developing these competencies is critical for enabling effective use of AI-assisted development tools.

6 Threats to Validity

We note several potential validity threats that could affect the interpretation of our findings. We list each threat in the following:

Self-reported experience classification. Participants were assigned to experience groups based on a self-classification question, introducing a risk of misclassification if participants overestimate or underestimate their own background. To reduce this risk the response options did not ask participants to subjectively rate their ability; instead, each option described a recognisable scenario anchored to concrete activities (e.g., the non-developer option specified "no formal programming training or professional experience, but having used AI tools to generate or modify code"; the professional option specified "professional or academic experience writing, maintaining, or reviewing code as part of a job, research, or formal development work"). This formulation allowed participants to map themselves onto the option closest to their actual experience rather than their self-perceived skill level. Furthermore, the experience self-classification question was administered twice in separate sessions approximately four weeks apart, allowing us to verify the stability of group assignments (Section 3.8); the matched-sample agreement of 74.0%, with the majority of changes drifting toward the middle "novice" category and only one participant moving between the two most distinct categories, indicates that misclassification at the boundaries of the most distinct groups is unlikely to drive the observed group differences.

Construct coverage and operationalisation. Our study constructs (motivations, experiences, perceived code quality, and QA practices) were derived from themes identified in our previous work [21], grounding the survey in real-world practice. While this supports content validity, some aspects of vibe coding may remain underrepresented. To check coverage, we included an optional open-ended response for each construct section and analysed the responses to identify any constructs not captured by our items; no additional dimensions emerged (Section 3.4).

Common method bias. All measures rely on participants' self-reports collected via a single survey at a single time point, increasing the risk of self-reporting and common method biases [59]. Participants may overstate the extent of careful QA practices or understate reliance on AI-generated

code, and consistent response tendencies may inflate the observed relationships between constructs. To reduce these effects, we (i) collected responses anonymously to lower social desirability pressure, (ii) separated the experience-classification stage from the main survey, (iii) organised constructs into distinct sections to reduce carry-over effects, (iv) used multi-item Likert scales per construct rather than single items, allowing patterns to be assessed at the construct level rather than relying on any one response, and (v) provided a consistent definition of vibe coding at the start of the survey to ensure shared understanding among participants. Where possible, items were phrased in behavioural rather than evaluative terms.

Perception–action gap interpretation. The perception–action gap is inferred from differences between reported awareness of AI-generated code limitations and reported QA behaviours. A key threat is that this pattern reflects self-reporting differences rather than actual behaviour (e.g., professionals may have underreported weaker practices, or less-experienced participants may have reported them more explicitly). To strengthen the interpretation, we examined the consistency of this pattern across multiple independent constructs (RQ2, RQ3, and RQ4) and across interaction styles. The recurrence of the pattern across different measures suggests that it is not an artefact of any single item or construct. Future work should validate this interpretation by examining actual behaviour, for example, through repository analysis or observational studies of how vibe coders evaluate and verify AI-generated code.

Sample composition and generalisability. Participants were recruited via Prolific, an online crowdsourcing platform. Although Prolific samples are more diverse than typical student samples [56], they may overrepresent individuals familiar with online platforms and survey-based tasks. The sample also reflects international participation with varied geographic and educational backgrounds, which may introduce cultural variation in attitudes towards AI code generation tools and in self-assessment. The inclusion of three distinct experience groups broadens coverage, but generalisation beyond similar populations should be made with caution.

Statistical robustness. The analysis involves multiple comparisons within each research question, increasing the risk of false positives. We applied the Benjamini–Hochberg procedure to control the false discovery rate, and we interpret results at the construct level, emphasising consistent patterns across related items and their effect sizes rather than isolated significant findings. We additionally excluded implausibly rapid responses using a completion-time threshold; a sensitivity analysis with a stricter threshold produced consistent patterns, supporting the robustness of the findings.

7 Conclusion

This paper presents a survey of vibe coding practices, comparing responses from 162 participants, representing three vibe coding user groups: non-developers, novice developers, and professional developers. We investigated vibe coding practices among those groups across four dimensions: motivations, experiences, perceived code quality, and QA practices followed.

Our analysis of the survey samples shows that prior programming experience shapes vibe coding selectively rather than uniformly. Reported experiences and perceptions of AI-generated code quality were largely consistent across groups, indicating a broadly shared general awareness of both the benefits and the limitations of vibe coding. Group differences instead concentrated on three areas: *why* participants engage in vibe coding, *how* they interact with AI code generation tools, and *how* they respond to issues in the code. Non-developers approached vibe coding as an accessibility-enabling AI-led practice. Novices treated it as an exploratory environment for learning. professionals adopted a more goal-oriented, iterative, and context-rich style, with greater use in work-related contexts. The clearest divergence appeared in QA practices, where less-experienced participants more often replaced debugging with reprompting using the same tools, and reported more frequent

breakdowns when AI-generated code was difficult to understand, whereas professionals verified outputs more consistently.

We interpret these observations as a potential *perception–action gap*. A general awareness of risks in AI-generated code appears broadly distributed across experience levels, but the capacity to act on that awareness through evaluation, debugging, and verification appears to remain experience-dependent. This reframes vibe coding as a *partially democratising* practice. Access to code production and high-level awareness of risks expands across user groups, yet evaluation and verification continue to differentiate experience levels. The risks of vibe coding, therefore, depend not only on the AI code generation tools, but also on the user's capability to act on what is already perceived. More broadly, this points to a partial decoupling between perceptual awareness and action-oriented competence in AI-assisted development.

Future work should examine the perception–action gap through behavioural studies that move beyond self-report, investigate how scaffolding mechanisms in AI code generation tools influence verification behaviour across experience levels, and evaluate educational interventions targeting the assessment of AI-generated code. As vibe coding continues to broaden participation in software creation, ensuring that the ability to evaluate what is produced keeps pace with the ability to produce it will be central to its safe and effective adoption.

References

[1] Alan Agresti. 2010. *Analysis of Ordinal Categorical Data* (2nd ed.). Wiley.

[2] Sri Haritha Ambati, Norah Ridley, Enrico Branca, and Natalia Stakhanova. 2024. Navigating (in) security of AI-generated code. In *2024 IEEE international conference on cyber security and resilience (CSR)*. IEEE, 1–8.

[3] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *2013 35th international conference on software engineering (ICSE)*. IEEE, 712–721.

[4] Sebastian Baltes and Stephan Diehl. 2018. Towards a theory of software development expertise. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 187–200.

[5] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.

[6] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the impact of early-2025 AI on experienced open-source developer productivity. *arXiv preprint arXiv:2507.09089* (2025).

[7] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289–300.

[8] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2022. Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools. *Queue* 20, 6 (2022), 35–57. doi:10.1145/3582083

[9] Rollin Brant. 1990. Assessing Proportionality in the Proportional Odds Model for Ordinal Logistic Regression. *Biometrics* 46, 4 (1990), 1171–1178.

[10] Virginia Braun and Victoria Clarke. 2006. Using thematic analysis in psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101.

[11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[12] Valerie Chen, Jasmyn He, Behnjamin Williams, Jason Valentino, and Ameet Talwalkar. 2026. Beyond the Commit: Developer Perspectives on Productivity with AI Coding Assistants. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26)*. Association for Computing Machinery, New York, NY, USA, 9 pages. Rio de Janeiro, Brazil, April 12–18, 2026.

[13] Jacob Cohen. 2013. *Statistical power analysis for the behavioral sciences*. Routledge.

[14] William Jay Conover. 1999. *Practical nonparametric statistics*. John Wiley & Sons.

[15] John W. Creswell and J. David Creswell. 2018. *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches* (5 ed.). SAGE Publications.

[16] Paul G. Curran. 2016. Methods for the Detection of Carelessly Invalid Responses in Survey Data. *Journal of Experimental Social Psychology* 66 (2016), 4–19. doi:10.1016/j.jesp.2015.07.006

[17] Paul Denny, James Prather, Brett A Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing education in the era of generative AI. *Commun. ACM* 67, 2 (2024), 56–67.

[18] Justin A DeSimone, Peter D Harms, and Alice J DeSimone. 2015. Best practice recommendations for data screening. *Journal of Organisational Behaviour* 36, 2 (2015), 171–181.

[19] K Anders Ericsson, Ralf T Krampe, and Clemens Tesch-Römer. 1993. The role of deliberate practice in the acquisition of expert performance. *Psychological review* 100, 3 (1993), 363.

[20] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering* 50, 9 (2024), 2254–2268.

[21] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2026. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook–a Grey Literature Review. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26)*. Association for Computing Machinery, New York, NY, USA, 9 pages.

[22] FedTec. 2025. Vibe Coding: Accelerating Service Delivery in the Government. https://fedtec.com/insights/vibe-coding-accelerating-service-delivery-in-the-government/. Accessed: 2026-05-07.

[23] Molly Q Feldman and Carolyn Jane Anderson. 2024. Non-expert programmers in the generative AI future. In *Proceedings of the 3rd Annual Meeting of the Symposium on Human-Computer Interaction for Work*. 1–19.

[24] Andy Field. 2024. *Discovering statistics using IBM SPSS statistics*. Sage Publications Limited.

[25] John Fox and Georges Monette. 1992. Generalized Collinearity Diagnostics. *J. Amer. Statist. Assoc.* 87, 417 (1992), 178–183.

[26] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2023. InCoder: A Generative Model for Code Infilling and Synthesis. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=hQwb-lbM6EL

[27] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security weaknesses of copilot-generated code in github projects: An empirical study. *ACM Transactions on Software Engineering and Methodology* 34, 8 (2025), 1–34.

[28] Matthias Galster, Muhammad Auwal Abubakar, Seyedmoein Mohsenimofidi, Christoph Treude, Jai Lal Lulla, and Sebastian Baltes. 2026. Configuring Agentic AI Coding Tools: An Exploratory Study. *arXiv preprint arXiv:2602.14690* (2026).

[29] Ruofan Gao, Amjed Tahir, Peng Liang, Teo Susnjak, and Foutse Khomh. 2025. A Survey of Bugs in AI-Generated Code. *arXiv preprint arXiv:2512.05239* (2025).

[30] Yuyao Ge, Lingrui Mei, Zenghao Duan, Tianhao Li, Yujia Zheng, Yiwei Wang, Lexin Wang, Jiayu Yao, Tianyu Liu, Yujun Cai, et al. 2025. A survey of vibe coding with large language models. *arXiv preprint arXiv:2510.12399* (2025).

[31] David M Groppe, Thomas P Urbach, and Marta Kutas. 2011. Mass univariate analysis of event-related brain potentials/fields I: A critical tutorial review. *Psychophysiology* 48, 12 (2011), 1711–1725.

[32] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*. ACM, Rio de Janeiro, Brazil.

[33] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. 2016. Usage, costs, and benefits of continuous integration in open-source projects. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 426–437.

[34] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[35] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. *arXiv preprint arXiv:2511.04824* (2025).

[36] Jason L Huang, Mengqiao Liu, and Nathan A Bowling. 2015. Insufficient effort responding: examining an insidious confound in survey data. *Journal of Applied Psychology* 100, 3 (2015), 828.

[37] Saki Imai. 2022. Is GitHub Copilot a Substitute for Human Pair-Programming? An Empirical Study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. Association for Computing Machinery, New York, NY, USA, 319–321. doi:10.1145/3510454.3522684

[38] Andrej Karpathy. 2025. There's a New Kind of Coding I Call "Vibe Coding". https://x.com/karpathy/status/1886192184808149383. Accessed: 2026-05-07.

[39] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–23.

[40] Amy J Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, et al. 2011. The state of the art in end-user software engineering. *ACM Computing Surveys (CSUR)* 43, 3 (2011), 1–44.

[41] Thomas D LaToza and Brad A Myers. 2010. Developers ask reachability questions. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE)*. 185–194.

[42] Dominik J. Leiner. 2019. Too Fast, Too Straight, Too Weird: Non-Reactive Indicators for Meaningless Data in Internet Surveys. *Survey Research Methods* 13, 3 (2019), 229–248. doi:10.18148/srm/2019.v13i3.7403

[43] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of AI programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1–13.

[44] Johan Linåker, Sardar Muhammad Sulaman, Rafael Maiani de Mello, and Martin Höst. 2015. *Guidelines for Conducting Surveys in Software Engineering*. Technical Report. Department of Computer Science, Lund University. https://lup.lub.lu.se/search/files/6062997/5463412.pdf

[45] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems* 36 (2023), 21558–21572.

[46] Noble Saji Mathews and Meiyappan Nagappan. 2024. Test-driven development and llm-based code generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1583–1594.

[47] Peter McCullagh. 1980. Regression Models for Ordinal Data. *Journal of the Royal Statistical Society: Series B* 42, 2 (1980), 109–142.

[48] Adam W. Meade and S. Bartholomew Craig. 2012. Identifying Careless Responses in Survey Data. *Psychological Methods* 17, 3 (2012), 437–455. doi:10.1037/a0028085

[49] Ivan Mehta. 2025. A Quarter of YC's Startups Have Codebases That Are Almost Entirely AI-Generated. https://techcrunch.com/2025/03/06/a-quarter-of-startups-in-ycs-current-cohort-have-codebases-that-are-almost-entirely-ai-generated. Accessed: 2026-05-07.

[50] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 142, 16 pages. doi:10.1145/3613904.3641936

[51] Brad A Myers, Amy J Ko, and Margaret M Burnett. 2006. Invited research overview: end-user programming. In *CHI'06 extended abstracts on Human factors in computing systems*. 75–80.

[52] Bonnie A Nardi. 1993. *A small matter of programming: perspectives on end user computing*. MIT Press.

[53] Nhan Nguyen and Sarah Nadi. 2022. An empirical evaluation of GitHub Copilot's code suggestions. In *Proceedings of the 19th International Conference on Mining Software Repositories (MSR)*. 1–5.

[54] Chris Parnin and Alessandro Orso. 2011. Are automated debugging techniques actually helping programmers?. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis (ISSTA)*. 199–209.

[55] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *Commun. ACM* 68, 2 (Feb. 2025), 96–105. doi:10.1145/3610721

[56] Eyal Peer, Laura Brandimarte, Sonam Samat, and Alessandro Acquisti. 2017. Beyond the Turk: Alternative platforms for crowdsourcing behavioral research. *Journal of Experimental Social Psychology* 70 (2017), 153–163.

[57] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).

[58] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with ai assistants?. In *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*. 2785–2799.

[59] Philip M Podsakoff, Scott B MacKenzie, Jeong-Yeon Lee, and Nathan P Podsakoff. 2003. Common method biases in behavioral research: a critical review of the literature and recommended remedies. *Journal of applied psychology* 88, 5 (2003), 879.

[60] James Prather, Brent N Reeves, Paul Denny, Brett A Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's Weird That It Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *ACM transactions on computer-human interaction* 31, 1 (2023), 1–31.

[61] Teade Punter, Marcus Ciolkowski, Bernd Freimut, and Isabel John. 2003. Conducting on-line surveys in software engineering. In *International Symposium on Empirical Software Engineering (EMSE)*. IEEE, 80–88.

[62] Brian J Reiser. 2018. Scaffolding complex learning: The mechanisms of structuring and problematizing student work. In *Scaffolding*. Psychology Press, 273–304.

[63] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at c: A user study on the security implications of large language model code assistants. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2205–2222.

[64] Christopher Scaffidi, Mary Shaw, and Brad Myers. 2005. Estimating the numbers of end users and end user programmers. In *2005 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC'05)*. IEEE, 207–214.

[65] Janet Siegmund, Christian Kästner, Sven Apel, Chris Parnin, Anja Bethmann, Thomas Leich, Gunter Saake, and André Brechmann. 2014. Understanding understanding source code with functional magnetic resonance imaging. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 378–389.

[66] Stack Overflow. 2025. 2025 Stack Overflow Developer Survey. https://survey.stackoverflow.co/2025/. Accessed: 2026-05-07.

[67] Florian Tambon, Arghavan Moradi-Dakhel, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Giuliano Antoniol. 2025. Bugs in large language models generated code: An empirical study. *Empirical Software Engineering* 30, 3 (2025), 65.

[68] Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, and Toby Jia-Jun Li. 2024. Developer behaviors in validating and repairing llm-generated code using ide and eye tracking. In *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 40–46.

[69] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. 1–7.

[70] Kurt VanLehn. 2011. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational psychologist* 46, 4 (2011), 197–221.

[71] Koen JF Verhoeven, Katy L Simonsen, and Lauren M McIntyre. 2005. Implementing false discovery rate control: increasing your power. *Oikos* 108, 3 (2005), 643–647.

[72] Yuvraj Virk and Dongyu Liu. 2025. Non-programmers Assessing AI-Generated Code: A Case Study of Business Users Analyzing Data. *arXiv preprint arXiv:2508.06484* (2025).

[73] Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2024. Investigating and designing for trust in ai-powered code generation tools. In *Proceedings of the 2024 ACM conference on fairness, accountability, and transparency*. 1475–1493.

[74] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 conference on Empirical Methods in Natural Language Processing*. 8696–8708.

[75] David Weintrop and Uri Wilensky. 2019. Transitioning from introductory block-based and text-based environments to professional programming languages in high school computer science classrooms. *Computers & Education* 142 (2019), 103646.

[76] Rainer Winkler and Matthias Söllner. 2018. Unleashing the Potential of Chatbots in Education: A State-of-the-Art Analysis. In *Academy of Management Proceedings*, Vol. 2018. Academy of Management, Briarcliff Manor, NY, 15903. doi:10.5465/AMBPP.2018.15903abstract

[77] Angela Yang. 2025. Noncoders Are Using AI to Prompt Their Ideas Into Reality. They Call It 'Vibe Coding'. https://www.nbcnews.com/tech/tech-news/noncoders-ai-prompt-ideas-vibe-coding-rcna205661. Accessed: 2026-05-07.

[78] Asli Yardim, Raphael Serafini, Nadine Jost, Anna-Marie Ortloff, Joshua Gabriel Speckels, and Alena Naiakshina. 2026. "The AI Tool Can't Make It Any Worse." Investigating Developers' Security Behavior with AI Assistants in a Password Storage Study. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. 1–33.

[79] Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778* (2023).

[80] Beiqi Zhang, Peng Liang, Xiyu Zhou, Aakash Ahmad, and Muhammad Waseem. 2023. Practices and Challenges of Using GitHub Copilot: An Empirical Study. In *Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering (SEKE)*. KSI Research Inc., 124–129. doi:10.18293/SEKE2023-077

[81] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.

[82] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2024. Measuring GitHub Copilot's impact on productivity. *Commun. ACM* 67, 3 (2024), 54–63.